Schreibmaschine

Manual for the text editor for macOS



For text and code.

Not an IDE, but a tool for writing: with a find bar, text commands, and a session that forgets nothing. Free, ad-free, and without data collection.

Version 1.0 · stefanradermacher.com

About this manual

Schreibmaschine is a text editor for macOS: first and foremost for plain text intended to be changed a lot, but just as useful for code. It doesn't want to be a development environment, just a tool for writing. In a few chapters, this manual describes what it can do and how to use it.

What makes Schreibmaschine special?

- Find and replace in a bar above the text, not in a window of its own
- A menu full of text commands and a command palette that finds every command
- A session of its own that never loses windows, tabs, or unsaved texts
- Syntax highlighting, indentation, brackets, and folding for code
- Multiple insertion points, bookmarks, and a split view
- Free and open source under the Apache License 2.0, no ads, and no data collection
- Apple frameworks only, no third-party components
- German and English

> “Schreibmaschine” [ˈʃʁaɪ̯pmaˌʃiːnə] is the German word for “typewriter”.

Documents and windows

Open text files by double-clicking them, through “File → Open…” (⌘O), or by dropping them onto the Schreibmaschine icon in the Dock or onto a Schreibmaschine window, whether on the text, the title bar, or the document list. Hold ⌥ while dropping onto the text to insert the file's path instead. ⌘N creates a new document.

Tabs or windows

New documents open as a tab in the front window or in a window of their own, as chosen under “Open new documents” in the settings. The macOS setting “Prefer tabs” plays no part in this. As everywhere on the Mac, tabs can be turned into windows and merged again.

Switching between documents

⌥⌘→ and ⌥⌘← go to the next and previous document, across all windows. ⌘1 to ⌘8 jumps to the first eight documents, ⌘9 to the last one. “Window → Documents” shows the order. ^⇥ switches between the tabs of one window, as usual.

The document list

^⌘S shows a list of all open documents on the left, grouped by window when there are several. A dot after the name marks unsaved changes; a click brings the document to the front.

Saving, encoding, and line endings

Schreibmaschine writes a file only when you save it with ⌘S, unless you turn on saving automatically (see below). It recognizes the text encoding and line endings when opening a file and keeps them when saving. Both appear in the status bar and can be changed there with a click. The common encodings are right in the menu; every other one that macOS knows is under “More Encodings”, grouped by script. If umlauts or other characters look wrong, “File → Reopen with Encoding” reads the file in another encoding.

If you like, Schreibmaschine tidies up when saving: under “Settings → Files → When saving”, it removes spaces and tabs at the ends of lines and ends the file with a line break. The line with the insertion point is left alone, and so are two spaces at the end of a Markdown line, which mean a line break there. Tidying up is a change like any other, and ⌘Z takes it back.

If another program changes an open file, such as git or a second editor, Schreibmaschine reloads it by itself as long as you have nothing unsaved in it; the insertion point and the scroll position stay where they were. If you have unsaved changes, a bar below the title bar asks: “Reload” takes the file, “Keep” stays with your text, which ⌘S then writes over the file, and “Compare” puts both side by side: take differences from the file over into your text and save the result. If an open file is deleted or moved, your text is kept and counts as unsaved; “Save As…” puts it somewhere again.

Saving automatically

With “Settings → Files → Save automatically”, Schreibmaschine writes changed files without ⌘S as well. “When switching away” saves as soon as the window or tab loses focus, Schreibmaschine goes to the background, or you close a document; a changed document is then saved on closing instead of asking. “When switching away and when pausing” also saves when you have not typed for 15 seconds. This is handy when another program reads the file, such as a compiler, a web server, or git.

Untitled documents are never saved automatically; they stay in the session. A file that another program has changed, deleted, or made read-only is left alone as well, and so is text with characters the file's encoding cannot hold; here, you decide with ⌘S. The marks in the ruler keep showing what changed since your last ⌘S, not since the last automatic save.

Earlier versions

Each time you open or save a file, by hand or automatically, Schreibmaschine keeps that state in its own area on your Mac. “File → Revert To” lists them: “Last Saved” and “Last Opened” at the top, the earlier versions with date and time below. Choosing a version replaces the text like an ordinary edit, so ⌘Z brings your text back. Hold down ⌥, and every entry becomes a comparison: the version appears beside your text, and you take over only what you need.

“Settings → Files → Keep earlier versions” sets how many versions each file keeps, 20 by default; none is kept longer than 14 days. Identical states are stored only once, and files over 10 MB not at all. “Clear History…” at the end of the menu removes the versions of the file; the file itself stays as it is. If you rename or move an open file, its versions go with it.

The session

Many Mac apps leave it to macOS to reopen what was open after a restart. Whether that works depends on system settings and on how the app was quit. Schreibmaschine therefore keeps a session of its own: all windows with their tabs, the files in them, and every unsaved text.

Nothing gets lost

Schreibmaschine keeps unsaved texts and changes all the time in its own area on your Mac, not in your files. That is why it never asks when you quit. At the next start everything comes back, marked as changed, even after a crash.

Starting with a file

Started without a file, Schreibmaschine restores the last session, opens a new document, or nothing, as set in the settings. When you open a file from the Finder instead, only that file appears. The session before is kept as the “previous session”: “Window → Session → Restore Previous Session” brings it back.

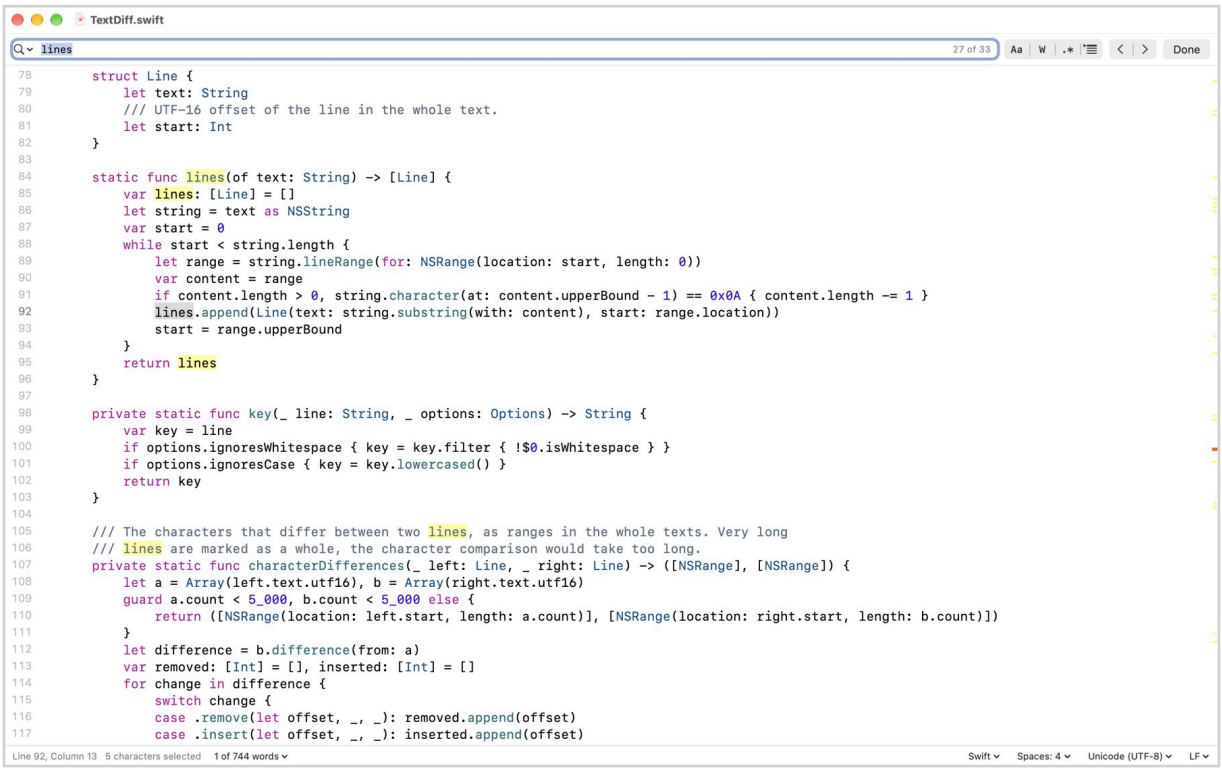
> “Window → Session” always shows what belongs to the session and what comes back at the next start.

Every file remembers its state

For every file, Schreibmaschine remembers the selection, the visible part, wrapping, line numbers, font size, bookmarks, folds, and a split view. Next time you open it, everything is as you left it. The file itself stays untouched.

Find and replace

⌘F opens the find bar above the text, ⌥⌘F the replace row as well. The search runs as you type. All matches are marked in the text and as lines at the right edge, and next to the field you see which match is selected.



The find bar with every match in the text

Finding the next match

↩ and ⇧↩ jump to the next and previous match, ⌘G and ⇧⌘G even with the bar closed. A selection becomes the search term when the bar opens; ⌘E takes it without opening the bar. The last 20 terms are in the menu of the magnifier in the search field, ↓ opens it as well. Esc closes the bar.

Even without the find bar, Schreibmaschine shows where a word occurs again: when you select a whole word, by double-clicking it, for example, its other occurrences get a light frame, and small lines at the right edge mark them in the whole document. ^G then adds the next occurrence to the selection. You can turn this off in the settings under “Appearance”.

Options

- Aa: match case
- W: whole words only
- .*: regular expressions; an invalid expression is reported in red
- Search in the selection only

Replacing

“Replace” replaces the current match and goes on to the next, “Replace All” replaces all of them in a single undo step. With regular expressions, $1 inserts groups, \n and \t stand for line break and tab, and a preview shows the result.

In all open documents

⇧⌘F searches all open documents. The matches appear in a sidebar, grouped by document; a click jumps to the spot. “Replace All” applies to all documents, and each can undo it on its own. Schreibmaschine deliberately does not search folders.

Go to line

⌘L opens the same bar for jumping: 120 goes to line 120, 120:15 also to column 15, and +10 and -10 jump relative to the current line.

Go to symbol

⇧⌘O shows the outline of the document: the headings in Markdown, the classes, functions, and the like in code, indented by their level. The entry the insertion point is in comes selected. A few letters filter the list, ↩ jumps to the entry, and Esc leaves everything as it was. In the command palette, an @ in front of the search does the same. The language definition says what belongs to the outline.

Reshaping text

The “Text” menu holds commands that reshape text in one step. Every command can be undone with ⌘Z, and ⌘R runs the last one again.

- Lines: duplicate, delete, move, join, split, sort, remove duplicates, reverse, shuffle, number, remove empty lines
- Paragraphs: rewrap and unwrap, indent and outdent, and mark as quote
- White Space: remove trailing spaces, convert between tabs and spaces
- Case: upper and lower case, also camelCase, snake_case, and kebab-case
- Columns: align at a character, format Markdown tables
- Encode: URL, Base64, and HTML, format or minify JSON

Without a selection

Without a selection, a command acts on what makes sense: sorting and encoding on the whole document, moving and indenting on the line, rewrapping on the paragraph, and changing case on the word at the insertion point.

The command palette

⇧⌘P opens the command palette. A few letters are enough: “jli”, for example, finds “Join Lines”. The palette knows every menu command and shows its menu path and shortcut; the commands used last are at the top.

Expanding the selection

⌥↑ expands the selection step by step: word, text in quotes, text in brackets, line, paragraph, everything. ⌥↓ goes back.

Several places at once

Multiple insertion points

⌘-click sets another insertion point, ⌘-drag adds another selection. Typing, deleting, ↩, ⇥, the arrow keys, and pasting act at all of them at once. If the clipboard holds as many lines as there are insertion points, each one gets its own line. Esc or a plain click goes back to a single insertion point.

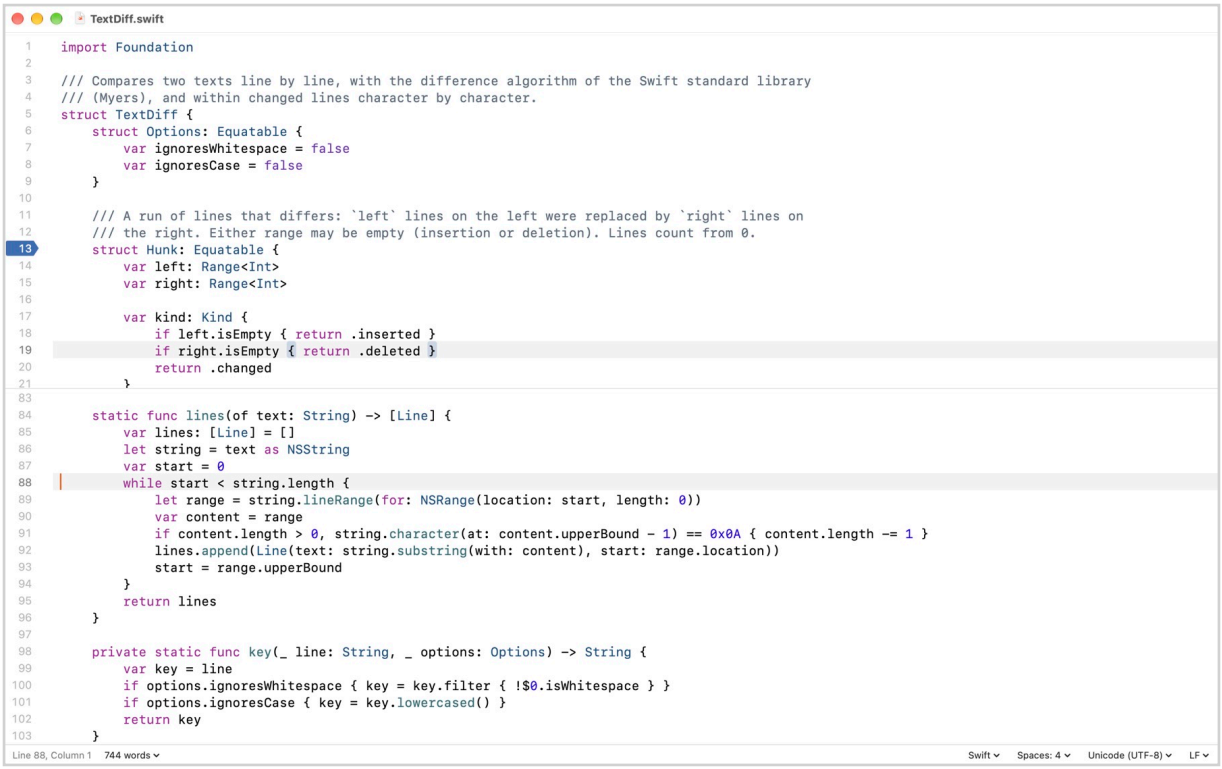
- ⌥-drag selects a block, with an insertion point or selection in every line
- ^G selects the word at the insertion point and then adds the next occurrence each time
- ⇧⌘L puts an insertion point at the end of every line of the selection
- ^⌘G turns all matches of the search into selections

Bookmarks

⌘B or a click on the line number sets a bookmark, a second one removes it. ⌥⌘↓ and ⌥⌘↑ jump to the next and previous one. “Edit → Bookmarks” lists them all with the text of their line. Bookmarks move with their line as you edit and are remembered for every file.

The split view

“View → Split Side by Side” (^⌘T) or “Split Top and Bottom” (^⌥⌘T) shows the text a second time, with its own selection and its own visible part. That way you can write in one place and keep an eye on another. ⌥⌘J goes to the other half, ^⇧⌘W closes the split.



The split view

Writing

Counting

On the left of the status bar is the number of words in the text, with a selection, for example “12 of 1,234 words”. A click on it shows all counts – words, characters, characters without spaces, lines, and paragraphs – and lets you choose which one the bar shows.

Typewriter mode

With “View → Typewriter Mode”, the line with the insertion point stays in the middle of the window, and the text moves past it, like the paper in a typewriter. Your eyes always stay in the same place.

Distraction-free mode

“View → Distraction-Free Mode” (^⇧⌘F) shows the text full screen in a narrow column, without status bar, sidebars, and line numbers. The width of the column is in the settings under “Writing”. The same command, or leaving full screen, brings everything back as it was.

Focus mode

With “View → Focus Mode”, only the paragraph with the insertion point stays strong; the rest of the text steps back. Strictly speaking it is the line with the insertion point; in running text, where every paragraph is one line, that is the whole paragraph.

> The three modes can be combined: in the distraction-free mode with typewriter and focus mode, the current paragraph stands in the middle of the screen, and everything else steps back.

Formatted Markdown

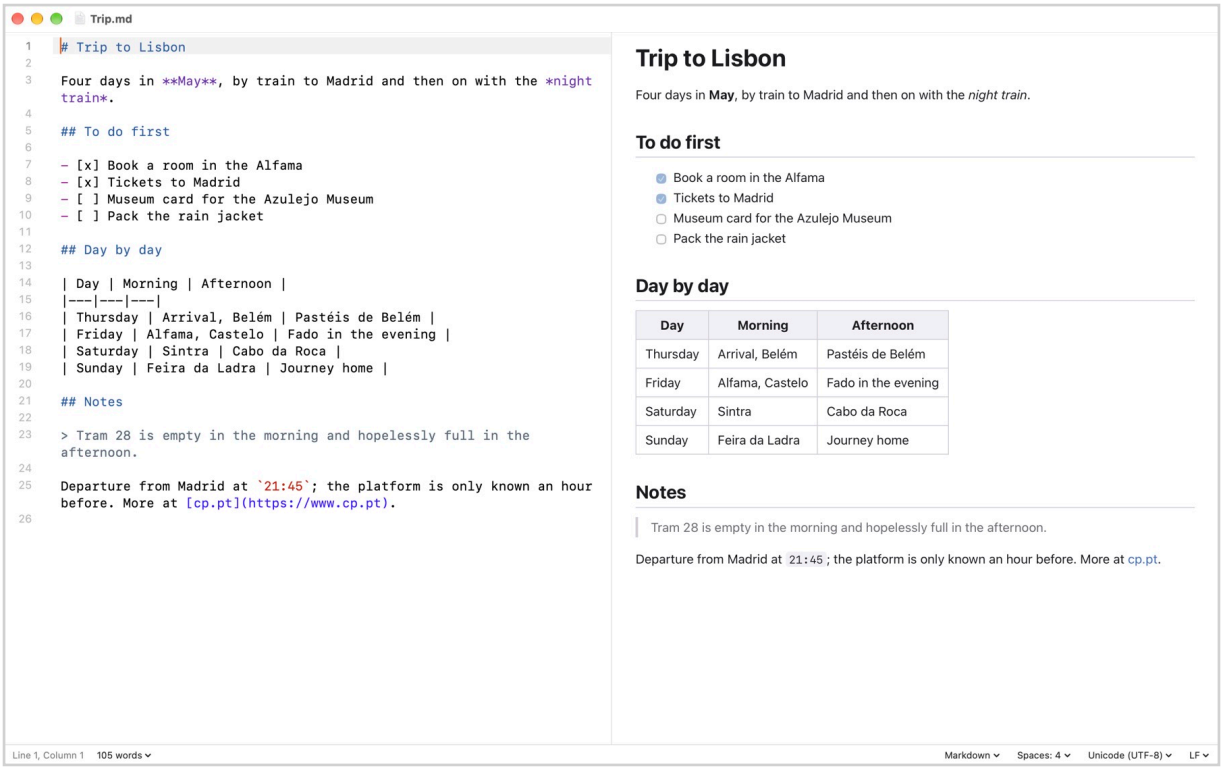
In Markdown files, Schreibmaschine shows the markup right in the text: headings are larger and bold, `**bold**` and `*italic*` appear bold and italic, `~~strikethrough~~` struck through, code has a light background, links are in the link color, and quotes get a bar at the edge. The marks themselves stay visible in a faint color, so you keep editing exactly the text that is in the file. Wrapped lines of list items and quotes start below the text, not below the mark in front of it. “View → Formatted Markdown” turns this off.

Lists and markup

↩ in a list item starts the next one, with the same indentation, the next number in numbered lists, and an empty box in task lists; ↩ in an empty item ends the list. Quotes go on the same way. ⇥ indents a list item, and ⇧⇥ outdents it. In “Text → Markdown”, “Bold” (^⌘B), “Italic” (^⌘I), “Code” (^⌘C), and “Strikethrough” put the marks around the selection or the word at the insertion point; a second time, they take them away again. “Link” (^⌘K) makes the selection the text of a link and selects the place for the address.

Markdown preview

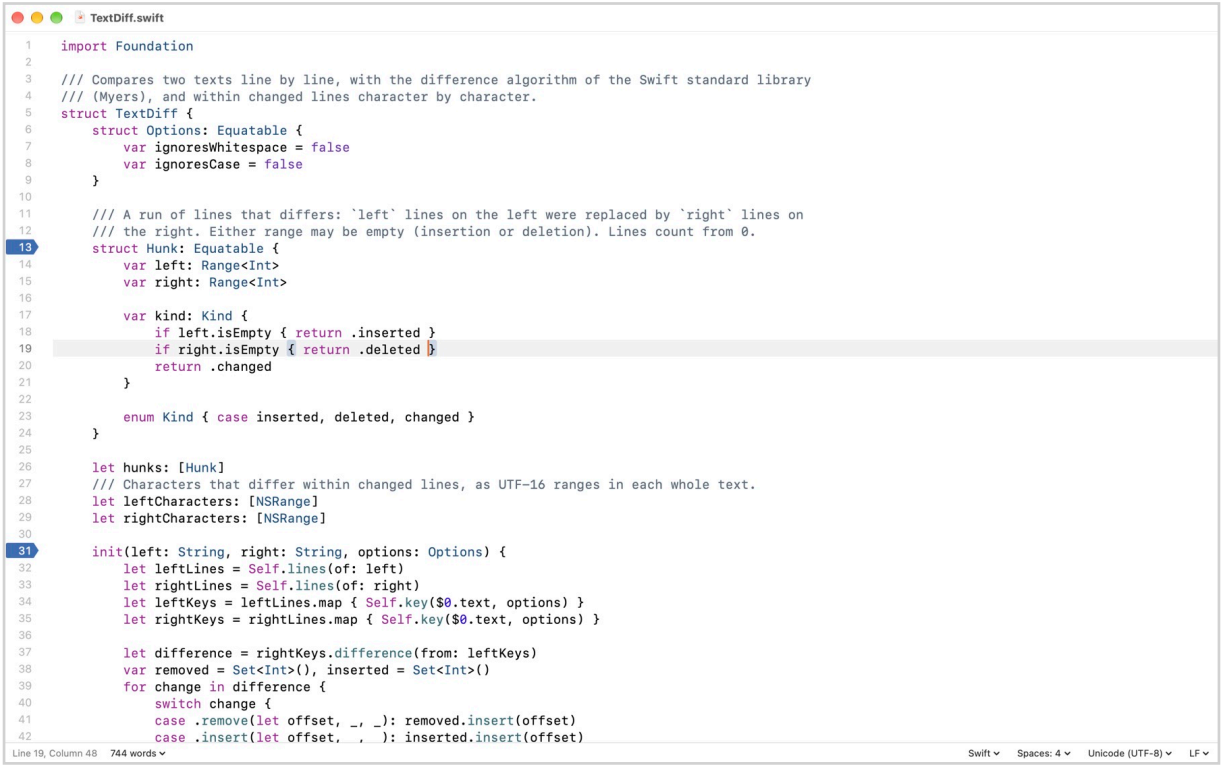
In a Markdown file, “View → Markdown Preview” (⌥⌘P) shows beside the text how it looks when set: headings, emphasis, lists and task lists, quotes, code blocks, tables, and links. The preview follows your typing after a short moment and scrolls along, aligned at the headings. Links open in the browser. The preview loads nothing from the internet; it shows images next to the file only if Schreibmaschine may read them, otherwise their description stands in their place.



A Markdown note with its preview

Code

Schreibmaschine recognizes the language of a file from its name, extension, or first line and colors it accordingly. It comes with Swift, Markdown, PHP, JavaScript and TypeScript, Python, JSON, YAML, INI and TOML, Shell, HTML, XML, CSS, SQL, and Lua. The language can be chosen under “Format → Language” or in the status bar.




```swift
import Foundation

/// Compares two texts line by line, with the difference algorithm of the Swift standard library
/// (Myers), and within changed lines character by character.
struct TextDiff {
    struct Options: Equatable {
        var ignoresWhitespace = false
        var ignoresCase = false
    }

    /// A run of lines that differs: `left` lines on the left were replaced by `right` lines on
    /// the right. Either range may be empty (insertion or deletion). Lines count from 0.
    struct Hunk: Equatable {
        var left: Range<Int>
        var right: Range<Int>

        var kind: Kind {
            if left.isEmpty { return .inserted }
            if right.isEmpty { return .deleted }
            return .changed
        }

        enum Kind { case inserted, deleted, changed }
    }

    let hunks: [Hunk]
    /// Characters that differ within changed lines, as UTF-16 ranges in each whole text.
    let leftCharacters: [NSRange]
    let rightCharacters: [NSRange]

    init(left: String, right: String, options: Options) {
        let leftLines = Self.lines(of: left)
        let rightLines = Self.lines(of: right)
        let leftKeys = leftLines.map { Self.key($0.text, options) }
        let rightKeys = rightLines.map { Self.key($0.text, options) }

        let difference = rightKeys.difference(from: leftKeys)
        var removed = Set<Int>(), inserted = Set<Int>()
        for change in difference {
            switch change {
            case .remove(let offset, _, _): removed.insert(offset)
            case .insert(let offset, _, _): inserted.insert(offset)
```

Swift with syntax highlighting, line numbers, and bookmarks

Indentation

Schreibmaschine recognizes from the text whether a file is indented with tabs or spaces and how wide. You can change it under “Format → Indentation” or in the status bar. ⇥ and ⇧⇥ indent and outdent a selection over several lines, ↩ keeps the indentation of the line before.

Comments and brackets

⌘/ adds or removes comment marks in front of the lines of the selection. ⌥⌘/ puts the selection, or the line if nothing is selected, into a block comment such as /* … */ or ; with the insertion point inside a block comment, it takes the comment away. Languages without block comments get line comments. The bracket next to the insertion point and its partner are highlighted, ⇧⌘M jumps to the partner.

Brackets and quotation marks close themselves: `(` brings its `)` right away, with the insertion point between them. If you type the closing bracket yourself, the insertion point just steps over it, and ⌫ between an empty pair deletes both. Before a word, no partner is added, and neither is one for an apostrophe in a word. With text selected, a bracket or quotation mark goes around the selection; in Markdown, this also works for `*`, `_`, and `~`. The language definition says which characters close themselves; in plain text, these are the brackets and `"`. You can turn this off in the settings under “Editing”.

Folding

Blocks that are indented deeper than the line before them can be folded. ^⌘← folds the block around the insertion point, ^⌘→ unfolds it, and with ⇧ it applies to everything. Arrows appear in the line number ruler as soon as the mouse is over it. A folded line ends in ⋯; a click on it unfolds the block.

Your own languages

Every language is a small YAML file with regular expressions. “Format → Language → Edit Definition of This Language” opens a copy of the definition that comes with the app, which you can adjust. New languages go into the folder that “Show Folder with Custom Languages” opens. Saving puts a change into effect right away; if the file has an error, Schreibmaschine names the line.

How a definition is built

- name, extensions, filenames, and interpreters: what the language is called and how it is recognized
- comment: the marks for line and block comments, used by ⌘/ and ⌥⌘/
- indent: the width of the indentation and whether to indent with tabs
- rules: the coloring rules, each with a scope such as keyword, string, or comment

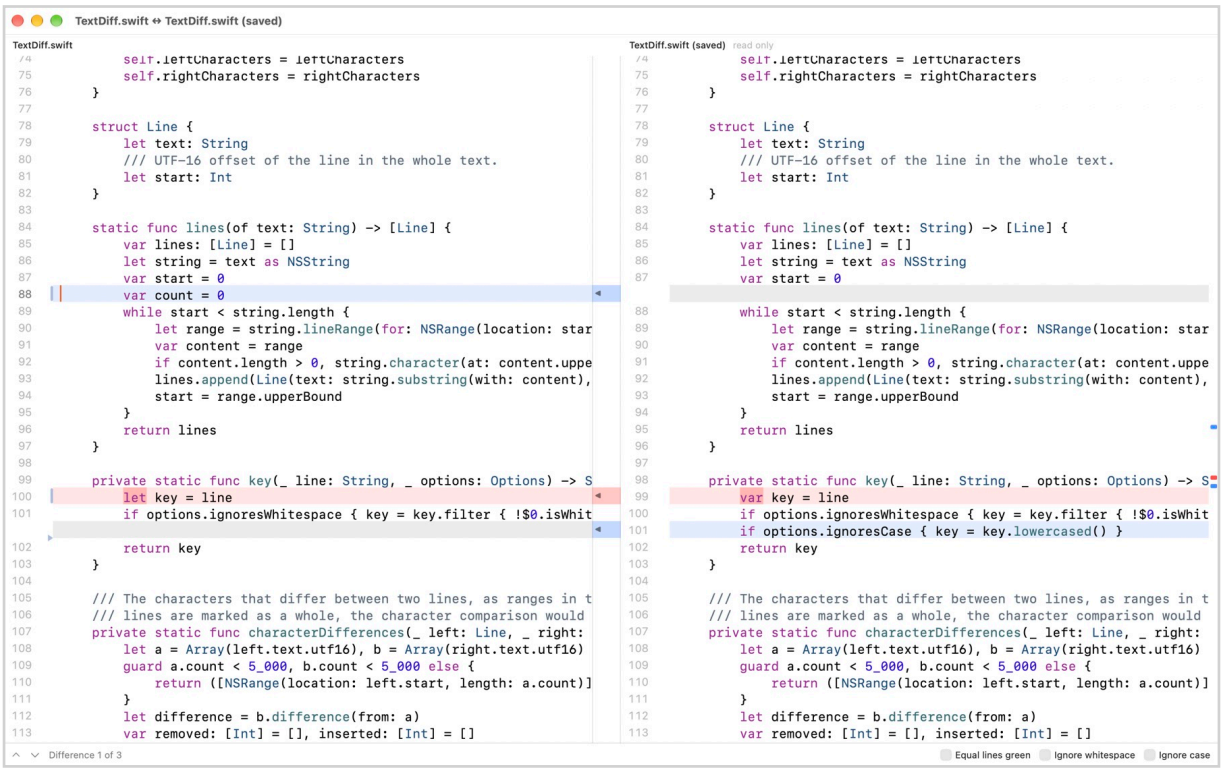
Rules

A rule names either words (words), a pattern within a line (match) or a region from begin to end, which may span several lines. In every line, the rule that matches earliest wins; comments and strings therefore belong at the top.

> Regular expressions are best put in single quotes, where the backslash is an ordinary character: '\d+' stays \d+.

Comparing

“File → Compare” puts two texts side by side: the document and its saved or an earlier version (with ⌥ under “File → Revert To”), the contents of the clipboard, another open document, or a file. The comparison opens in a window of its own, which you can drag to the tabs like any other.



A document compared with its saved version

Both sides scroll together. Where lines are missing on one side, Schreibmaschine leaves room, so lines that belong together stand at the same height. Neither side counts as the older one: lines that exist on only one side are blue, whichever side; lines that differ are red, with the differing characters marked more strongly. If you like, equal lines are green as well (“Equal lines green” at the bottom right); otherwise, they stay white. The bar at the right edge shows all differences in the whole text; a click jumps there.

Working through the differences

^⌘↓ and ^⌘↑ or the arrows at the bottom left jump to the next and previous difference; beside them it says “Difference 3 of 12”. Both sides are ordinary editors, and the comparison follows your typing. A click on the left half of a block in the middle strip takes it over from right to left, a click on the right half vice versa; ⌘Z undoes it. ⌘S saves the side that holds the insertion point. The saved version and the clipboard are read only.

Options and result

At the bottom right you can ignore whitespace and case. “Copy Differences” and “Differences as New Document” pass the comparison on as a unified diff, the way `diff -u` writes it.

Scripts

You can write your own transformations as scripts in any language that runs on the Mac: shell, Perl, Python, Ruby, Swift, or whatever else is installed. They live in the scripts folder, which “Scripts → Open Scripts Folder” shows in the Finder, and appear in the Scripts menu and in the command palette. New scripts are there right away. Subfolders become submenus; a number before the name only sets the order: “01 Sort.sh” is called “Sort” in the menu. “Install Example Scripts…” puts three examples into the folder.

A script gets text on its standard input and gives text back on its standard output. It has to be executable (in Terminal: chmod +x) and start with a line such as #!/bin/zsh. Without further details, it gets the selection, or the whole document if nothing is selected, and its result replaces exactly that. So two lines with sort already make a sort command of your own. It gets the path of the file as its first argument.

Input and output

Two lines at the beginning of the script, in its comment syntax, change that:

- @input selection: the selection, or the document if nothing is selected (the default)
- @input document: always the whole document
- @input none: nothing
- @output replace-selection: replaces what went in (the default)
- @output replace-document: replaces the whole document
- @output insert: inserts after the selection
- @output new-document: opens a new document
- @output copy: puts the result on the clipboard
- @output none: discards the result

Replacing text with a result is one step that ⌘Z undoes. If a script ends with an error, Schreibmaschine shows what it wrote to its error output. If the text changes while a script runs, Schreibmaschine does not put the result in but offers it as a new document.

> Scripts run outside Schreibmaschine's sandbox, with your rights; they can do anything a program in Terminal can. Only put scripts you trust into the folder.

View and settings

Line numbers (⌥⌘L), line wrapping (^⌘W) and invisible characters (⌥⌘I) apply to each window on its own. Beside the line numbers, a narrow bar shows which lines are new or changed since the last save with ⌘S, and a small wedge where lines were deleted; after ⌘S, the marks disappear. Whether and in which colors they appear is set under "Settings → Appearance → Changes in the ruler". ⌘+ and ⌘- change the font size, ⌘0 resets it.

Scrolling and jumping

Page Up and Page Down scroll by the height of the window and take the insertion point along. Home and End go to the start and end of the line, ⌘Home and ⌘End (like ⌘↑ and ⌘↓) to the start and end of the document; with ⇧ they select. Home and ⌘← first go to the end of the indentation, and a second time to the very start of the line. If you prefer the macOS behavior, where these keys only scroll, switch it off in the settings under Editing.

The status bar

At the bottom of the window are line and column, the size of the selection, language, indentation, text encoding, and line endings. Everything except the position can be changed with a click.

Settings

- General: what happens when the app starts without a file, tabs or windows, and the default app for text and code
- Files: saving automatically, how many earlier versions are kept, tidying up when saving, and encoding and line endings for new documents
- Appearance: font, line height (from +0% to +100% over what the font needs, +20% by default), defaults for new windows, highlighting of syntax, Markdown, the current line, and occurrences of the selected word, and changes in the ruler
- Editing: tab width and indentation, closing brackets automatically, Page Up/Down, Home, and End, and the wrap column
- Writing: typewriter mode, focus mode, and the width of the text column in distraction-free mode

To have text files open in Schreibmaschine with a double-click, “General → Default app → Make Default” makes it the default for plain text, Markdown, JSON, YAML, XML, CSS, JavaScript, Swift, Python, PHP, shell scripts, SQL, Lua, TOML, and INI, as well as diff. macOS asks for each format separately. “Customize…” lists all formats with the app that opens them now and sets each one on its own, including HTML, CSV, and logs, which usually stay with the browser, a spreadsheet, and Console. “Reset” gives the formats back to the apps they had before. After a few days, Schreibmaschine also asks once itself, in a bar below the title bar, at most twice.

The default font is SF Mono, the monospaced font of macOS. Because macOS does not list it in the font panel, the “Default” button leads back to it.

Keyboard shortcuts

All shortcuts are ordinary menu commands. They can be changed in System Settings under “Keyboard → Keyboard Shortcuts → App Shortcuts”.

Find, Find and Replace	⌘F, ⌥⌘F
Find Next, Find Previous	⌘G, ⇧⌘G
Find in all open documents	⇧⌘F
Go to Line, Go to Symbol	⌘L, ⇧⌘O
Command palette	⇧⌘P
Repeat the last text command	⌘R
Duplicate line, delete line	⌘D, ⌘⌫
Move lines	⌥⇧↑, ⌥⇧↓
Join lines	⇧⌘J
Indent, outdent	⌘], ⌘[
Line, block comment, matching bracket	⌘/, ⌥⌘/, ⇧⌘M
Expand, shrink selection	⌥↑, ⌥↓
Add the next occurrence	^G
Select all matches	^⌘G
Insertion point in every line of the selection	⇧⌘L
Add bookmark, next, previous	⌘B, ⌥⌘↓, ⌥⌘↑
Fold, unfold (with ⇧: everything)	^⌘←, ^⌘→
Split side by side, top and bottom	^⌘T, ^⌥⌘T
Go to other half, close split	⌥⌘J, ^⇧⌘W
Document list	^⌘S
Next, previous difference in a comparison	^⌘↓, ^⌘↑
Markdown preview, distraction-free mode	⌥⌘P, ^⇧⌘F
Markdown bold, italic, code, link	^⌘B, ^⌘I, ^⌘C, ^⌘K
Next, previous document	⌥⌘→, ⌥⌘←
Documents 1 to 8, last document	⌘1 – ⌘8, ⌘9
Bigger, smaller, actual size text	⌘+, ⌘-, ⌘0

Line numbers, wrapping, invisible characters	⌥⌘L, ^⌘W, ⌥⌘I

Questions and answers

Is Schreibmaschine a development environment?

No, and it will stay that way. There are no projects, no auto-completion, and no building or running. Schreibmaschine is a tool for writing and reshaping text, which may well be code.

Where do my unsaved texts go when I quit?

They stay in the session and come back at the next start. Files are only written with ⌘S, unless you have turned on saving automatically. An untitled document that you close yourself, on the other hand, asks whether it should be saved; closing is a deliberate act, quitting is not.

I opened a file from the Finder, and my other windows are gone.

When Schreibmaschine starts because of a file, it shows just that file. “Window → Session → Restore Previous Session” brings the other windows back.

How large may a file be?

As large as your memory allows. A log file of 100 MB with a million lines opens in about a second, and jumps to any place are immediate. Lines of more than 10,000 characters, as in minified JSON, stay uncolored. If a file contains a line of more than 100,000 characters, its window opens with line wrapping, because such a long line costs a lot of time and memory unwrapped; ^⌘W turns wrapping off all the same.

Why doesn't Schreibmaschine search folders?

Because it has no notion of projects. The search in all open documents covers what you are working on; for folders there are the Finder and command-line tools.

What do the tips get me?

Nothing but my thanks. They unlock no features; Schreibmaschine is and stays completely free.